

Usage Manual

How to use the refactored pipeline of the GitHub repository

Companion material of the article “Assessing a Mathematical Model of Syllable Production via CTW Alignment with EMA Data”

by F. Berthommier (Univ. Grenoble Alpes, CNRS, Grenoble INP, GIPSA-lab, France)

(Interspeech 2026 submission 2479, resubmitted to arXiv)

F. Berthommier – Gipsa-Lab, CNRS, University Grenoble Alpes

September 2026

Finalization banner. The cross-references between the three repositories (GitHub, Zenodo, arXiv) are being finalized. Insert here the Zenodo DOI, the arXiv identifier and the official Interspeech-style reference once available (see `docs/FINALIZATION-CHECKLIST.md`).

1 Purpose and scope

This manual describes how to *use* the functions of the repository after the refactoring of the original research scripts. Every original script of the legacy `SynthsylShort/` workspace has been split into conceptually distinct, individually callable functions, distributed across the `src/`, `figures/`, `synthesis/` and `video/` directories, and wired into a master driver, `run_all.py`, at the repository root. The correspondence between the original scripts and the new modules is documented in `docs/mapping.md`; the data-flow dependencies are documented in `docs/PIPELINE.md`.

With this repository, a fresh clone regenerates *all* the numerical results of the article and of its supplement from the raw corpus (`data/mat/`, `data/wav/`): the intermediate articulatory and formant tracks, the nine article/supplement figures, the statistical analyses (per-parameter correlations, paired *t*-tests with Holm correction), a demonstration syllable-synthesis corpus and the MP4 stimulus videos. An automated test suite (`tests/test_reproducibility.py`) compares the regenerated values against the archived ones and quantifies the reproducibility guarantees (Section 9).

2 Installation

Python ≥ 3.9 is required. Video reproduction needs `ffmpeg`; if it is not installed on your system, the `imageio-ffmpeg` package (included in `requirements.txt`) provides a bundled binary that the pipeline locates automatically.

```
git clone <REPOSITORY-URL> % TODO-FINALIZATION: insert final URL
cd <repository>
python -m venv .venv
.venv\Scripts\activate % Windows (Linux/macOS: source .venv/bin/activate)
pip install -r requirements.txt
python -m pytest tests/test_reproducibility.py --collect-only % sanity check
```

3 The master driver: run_all.py

All stages can be driven from the repository root. Each stage is idempotent (outputs already present are skipped; add `--force` to recompute), logged to the console and to `results/logs/run_all.log`, and stops with a clear message if an input directory is missing. All randomness is seeded inside the generators (the condition B model pairing uses seed 126; the LPC synthesis reseeds NumPy per stimulus), so runs are deterministic on a fixed environment.

```
python run_all.py --stage data % mat/wav -> copyparam, copywav, copyformants
python run_all.py --stage ctw % -> paramodelctw(random), modelctw(random)
python run_all.py --stage figures % the 8 figure scripts (9 PNGs) + statistics
python run_all.py --stage synthesis % demonstration syllable corpus
python run_all.py --stage videos % MP4 chain (ffmpeg required)
python run_all.py --stage all % everything, in this order
```

Generators write under `results/regenerated/data` by default (same tree structure as `data/`); the archived `data/` directory is never modified by the pipeline. Common options:

```
--out-root DIR % where the regenerated data tree is written
--force % recompute even if outputs already exist
```

Measured runtimes (one core): `data` ~45 min, `ctw` ~75 min, `figures` ~1 min, `synthesis` ~15 s, `videos` ~1 min; the full `--stage all` chain takes about two hours. The dominant cost is the VLAM formant synthesis (~35 s per stimulus, 63 stimuli, three passes across the stages).

4 Naming conventions

All stimuli follow the YOX rule: $X \in [10, 16]$ is the item index and reference model number, $Y \in [2, 10]$ the speaker/condition index; the two-digit X is always the last two characters of the name (e.g. 2012, 10016). The archived files are prefixed by set:

CP_*.npz	reference (EMA-derived) 6-parameter articulatory tracks, shape (1000, 6);
CF_*.npz	formant tracks (F1–F3) of the reference-driven VLAM synthesis, shape (1, 1000, 3);
CW_*.wav	synthesized reference audio (20 kHz, 16-bit PCM);
PM_*.npz / PMR_*.npz	CTW-aligned parameter tracks of condition A / condition B, shape (7, 1000) (row 6, Hyoid, zeroed);
MPF_*.npz / MPFR_*.npz	dictionaries {'fval', 'Pv'}: formants (3, 1000, 1) and parameters (7, 1000) of condition A / condition B;
P0<X>.npz	Maeda reference model parameters (dictionaries {'fval', 'Pv', 'dur'}).

5 Using the data-generation functions (src/)

The four generators of the intermediate data sets expose one importable function per conceptual operation. They can be used as CLIs (shown first) or imported into your own scripts.

5.1 Raw EMA to reference parameters

```
python src/batch_parameters.py --out-root results/regenerated/data
```

- `load_ema_reduce(mat_path)` — load one MATLAB EMA file (`VV_*.mat`, shape $(T, 27)$), extract the 14 validated coil channels (X: indices 2–8, Y: 11–17).

- `reduce_to_6vars_mat(data14)` — reduce the 14 channels to the 6 Maeda-like parameters (Jaw, Body, Dorsum, Tip, LipP, LipH); returns shape $(T, 6)$.
- `zero_phase_lowpass_filter(data)` — 8 Hz zero-phase (forward-backward) Butterworth low-pass along the last axis.
- `process_utterance(fname2, fname1, out_dir=None)` — complete chain for one utterance: decimation 5:1, filtering, z -normalization, renormalization with the mean/standard deviation of model P0<X>, and save as `CP_<fname2>.npz`. With `out_dir=None` the file is written to the canonical `data/copyparam/` location.

5.2 Reference audio and formants (VLAM synthesis)

```
python src/make_reference_tracks.py --names 2012,8015 --out-root results/regenerated/
data
```

- `synthesize_track(name, copyparam_dir, wav_dir)` — VLAM/LPC synthesis of one stimulus: the 6 reference parameters (Hyoid appended as zero) drive the articulatory model, the energy envelope of the real recording `vv_<name>.wav` (Hilbert envelope, 8 Hz low-pass, empirical 0.1 threshold) shapes the glottal excitation. Returns `(sig, fval)`.
- `save_track(name, sig, fval, out_root)` — write `CW_<name>.wav` (20 kHz, PCM_16, the archived subtype) and `CF_<name>.npz` (`fval.T`).
- `process_one(name, ..., force=False)` — synthesize and save one stimulus; skips it if both outputs already exist.

5.3 CTW alignment, conditions A and B

Both conditions share the helpers of `src/ctw_tracks.py`:

- `stimulus_names()` — the list of the 63 stimulus names, in the original (X outer, Y inner) loop order.
- `load_model_track(x_index)` — parameters of model P0<X>, shape $(7, T_1)$.
- `load_natural_track(name, copyparam_dir)` — natural parameters of one utterance, shape $(6, T_2)$.
- `align_model_to_natural(model_track, natural_track)` — Canonical Time Warping of the model onto the natural track (reconstruction on the T_2 time grid, gap interpolation), followed by the 8 Hz zero-phase smoothing; returns the aligned track of shape $(7, T_2)$. This is exactly the computation displayed by the Figure S1a/S1b scripts.

The condition scripts wrap these helpers with their pairing rules and the shared formant conversion:

```
python src/ctw_condition_a.py --names 2012 --out-root results/regenerated/data
python src/ctw_condition_b.py --names 2012 --out-root results/regenerated/data
```

- `ctw_condition_a.condition_a_model_index(name)` — condition A pairs each utterance Y0X with its own model P0<X> (X = last two characters of the name).
- `ctw_condition_b.pick_random_models(seed=126)` — condition B pairing: one `random.choice` per stimulus among the models $X2 \neq X$, replicating call for call the seeded generator of `figures/figs1b*.py`.

- `generate_condition_a_track(name, copyparam_dir) / generate_condition_b_track(name, copyparam_dir)` — aligned parameter track ($7, T_2$) of one stimulus.
- `process_one(name, copyparam_dir, out_root, force)` — align, convert and save the PM_/MPF_ (condition A) or PMR_/MPFR_ (condition B) pair of files.

5.4 Parameter-to-formant conversion

- `formant_conversion.convert_aligned_parameters_to_formants( param_track)` — evaluate the VLAM model frame by frame on a $(7, T)$ aligned track (non-finite values cleaned, Hyoid row zeroed) and return the legacy dictionary `{'fval': (3, T, 1), 'Pv': (7, T)}`. This is the exact computation of `video/vlam_audio_maker_v2.py`, factored out so both condition pipelines and the video chain share it.

5.5 Low-level analysis helpers

- `ctw.apply_ctw_path(ref_ema, target_ema)` — CTW path (tslearn) and the aligned reference reconstructed on the target time grid (per-frame averaging, linear interpolation of gaps); returns `(aligned, path_matrix)`.
- `correlations.compute_correlation_simple(pv1, pv2, labels=None, verbose=False)` — per-column Pearson correlations computed manually (sample standard deviations, `ddof=1`, matching the original research code); constant columns get 0.
- `stats_tests.paired_ttests_holm(corr_matrix, indices, param_names, row_label)` — paired t -tests between the A/B columns of each parameter with Holm correction ($\alpha = 0.05$); prints the article table and returns the list of result dictionaries `(mean_a, t_stat, p_raw, p_holm, significant, ...)`.
- `plots.plot_ctw_path(path_matrix, color, ax)` and `plots.plot_articulatory_parameters(...)` — matplotlib helpers used by the figure scripts.

6 Reproducing the figures

Each figure is generated by one self-documented script reading only the archived `data/` directory (see `docs/mapping.md` for the correspondence with the original research scripts):

```
python figures/fig2a_articulatory_correlations.py % Figure 2a (article)
python figures/fig2b_formant_correlations.py % Figure 2b (article)
python figures/fig3_formant_trajectories.py % Figure 3 (article)
python figures/figs1a_ctw_paths_condition_a.py % Fig. S1a (supplement)
python figures/figs1b_ctw_paths_condition_b.py % Fig. S1b (supplement)
python figures/figs3_articulatory_param_trajectories.py % Fig. S3 (2 PNGs)
python figures/figs4_ctw_model15_on_model12.py % Fig. S4 (supplement)
python figures/figs5_ctw_alignment_three_way.py % Fig. S5 (supplement)
```

The nine PNGs are written to `results/figures/` at 300 dpi, and the printed statistical tables (correlations, paired t -tests, Holm correction) are part of the reproducible record: they must match the numbers of the article and the reference file `tests/reference_stats.json`. Note that the Figure S1b script also (re)saves its aligned tracks into `data/paramodelctwrandom/`; this write is deterministic (seed 126) and byte-identical to the archive.

7 Reproducing the syllable synthesis

The `synthesis/` package refactors the legacy interactive synthesizer into twelve modules (configuration, Tau-model trajectories, VLAM articulatory model, tube acoustics, LPC synthesis, pipeline, input/output). The entry point is a CLI:

```
python synthesis/run_synthesis.py --text "ba.du abi" --stem essai
python synthesis/run_synthesis.py --text "ga.bi" --no-video --no-plots --no-
spectrogram
```

Input words combine the vowels u o 0 a e i y E, the consonants b d g, and dots between syllables. Outputs in `results/synthesis/`: `<stem>.wav` (20 kHz), `<stem>.npy` (dictionary `fval/Pv/dur`) and, unless disabled, the vocal-tract animation `output.avi` and the parameter, formant and spectrogram plots. Omitting `--text` enters the interactive prompt of the original script. The pipeline reads no data files.

8 Reproducing the MP4 videos

The video chain renders the VLAM midsagittal animation of stimulus 2012 and assembles the article videos. All of it is automated by `python run_all.py --stage videos`; the individual scripts are:

```
% Elementary renders (XVID AVI at 100 fps, then ffmpeg audio mux):
python video/vlam_video_maker.py --source copyparam --mp4 ../results/videos/CondA.mp4
python video/vlam_video_maker.py --source modelctwrandom --mp4 ../results/videos/CondB
.mp4
python video/vlam_video_maker.py --source copyparam --mp4 ../results/videos/CondRef.
mp4
% Compositing chain (each step consumes the previous outputs):
python video/superpose_condition_a_reference.py % CondRef + ConDA -> mixARef.mp4
python video/superpose_condition_b_reference.py % CondRef + CondB -> mixBRef.mp4
python video/superpose_conditions_a_b.py % ConDA + CondB -> mixAB.mp4
python video/side_by_side_composites.py % mixARef | mixBRef -> mixABRef.mp4
```

Every script accepts `--input-dir`, `--output-dir` and `--ffmpeg` so the chain can run entirely inside `results/videos/` without touching the archived `data/videos/`. `video/vlam_audio_maker.py` (reference synthesis, `CP_*` → `CW_/CF_*`) and `video/vlam_audio_maker_v2.py` (condition B conversion, `PMR_*` → `MPFR_*`) remain available with their own `--out-dir` options; the `src/` generators of Section 6 are their pipeline-wired counterparts.

9 Reproducibility guarantees and tests

`tests/test_reproducibility.py` (pytest, ~20 min on one core) runs three families of checks on a fixed subsample of five stimuli:

1. **Archive equality.** The generators are compared with the archived data at two levels. *Identity*: fed with the archived `CP_*` inputs, the generators reproduce the archives to machine precision (parameters $\leq 4 \cdot 10^{-14}$, formants $\leq 3 \cdot 10^{-11}$ Hz, waveforms within 1 least-significant bit of the 16-bit PCM). *End-to-end*: restarting from the raw `data/mat/` corpus, the regenerated parameters deviate by $\leq 2 \cdot 10^{-5}$ (a NumPy/SciPy-version drift on the float32 EMA reduction, not a methodological difference); the CTW path can then flip on a few near-tie frames, so the tests combine a per-frame maximum tolerance with a bound on the fraction of affected frames ($\leq 5\%$ above 0.05 parameter units / 10 Hz; measured maxima $\leq 2.8\%$). All tolerances are documented in the test file and justified in `docs/PIPELINE.md`.

2. **Statistics.** The Figure 2a/2b analyses recomputed from `data/` (means, standard deviations, t -statistics, raw and Holm-corrected p -values, significance flags) must match `tests/reference_stats.json` to 10^{-9} . That file is produced once by `tests/generate_reference_stats`
3. **Figures.** Each of the eight figure scripts must run and produce a non-empty PNG.

```
python -m pytest tests/ -q % full suite (~20 min)
python -m pytest tests/ -q -k statistics % statistics only (~10 s)
python tests/generate_reference_stats.py % regenerate the reference numbers
```

10 Repository layout

<code>run_all.py</code>	Master driver of the five stages.
<code>data/</code>	All data: <code>mat/</code> raw EMA (Kasper et al., third-party license); <code>wav/</code> , <code>copywav/</code> audio; <code>maeda/</code> model parameters; <code>copyparam/</code> , <code>copyformants/</code> reference tracks; <code>paramodelctw(random)/</code> , <code>modelctw(random)/</code> CTW products; <code>videos/</code> archived MP4s.
<code>src/</code>	Reusable library (CTW, correlations, statistics, plots) and the intermediate-data generators of Section 6.
<code>figures/</code>	One script per article/supplement figure.
<code>synthesis/</code>	Short-syllable synthesis package.
<code>video/</code>	Audio/video stimulus makers and compositing chain.
<code>tests/</code>	Reproducibility test suite and reference statistics.
<code>docs/</code>	This manual, <code>documentation.pdf</code> , <code>PIPELINE.md</code> (data-flow map), <code>mapping.md</code> (script correspondence), finalization checklist.
<code>results/</code>	Outputs regenerated by the scripts (not tracked): <code>figures/</code> , <code>videos/</code> , <code>synthesis/</code> , <code>regenerated/data/</code> , <code>logs/</code> .

11 Licenses and citation

Code, figures, documentation and videos: MIT (Berthommier, Gipsa-Lab). EMA data: MIT third party, attributed to Kasper et al. (`LICENSES/LICENSE-EMA-data.md`); the data were kindly transmitted to the author personally by the authors of that publication (personal communication, 2026). See `CITATION.cff`. **TODO-FINALIZATION:** insert the Zenodo DOI, the arXiv identifier and the official Interspeech-style reference once the three deposits are finalized.

References

1. J. Kasper, C. A. Kell, P. Perrier, “An informational account of anticipatory coarticulation – theoretical considerations and empirical plausibility”, *Journal of Neurolinguistics*, vol. 78, p. 101298, 2026. doi:10.1016/j.jneuroling.2025.101298.